Beyond Context Compression: State Composition and Recent History in Coding Agents

Alex Markov-Buturskiy

Abstract

Structured state offers a way to bound a coding agent's active context without carrying its full interaction history. Whether this also reduces the cost of solving tasks depends on what the replacement memory preserves. We evaluate SKILL.state-inspired context policies on Terminal-Bench, varying state ownership, retained evidence, recent history, update protocol, and schema. Model-written state reduces the median peak-prompt ratio to 0.47 on shared successes, yet raises the cost ratio to 2.57 and lowers overall task success from 61.3% to 34.9%. An exploratory comparison of hybrid memory, which separates runtime records from model-written interpretation, associates richer execution records with better success and lower overhead than file references alone. Further enrichment does not remove the full-history deficit. Retaining recent interactions gives the clearest recovery: model-written and hybrid policies reach 59.0% and 48.3%, against their respective controls of 61.3% and 51.7%. The economic outcomes differ: cost per observed success falls below the control for model-written memory but remains above it for the hybrid variant. The tested schema and protocol changes do not produce comparable recovery. These results support evaluating structured memory through the evidence it retains and the work needed to finish a task, rather than compression alone. Recent-history retention is a measured partial remedy; the incremental benefit of state over the same history window remains open.

1 Introduction

A coding agent accumulates evidence as it works. A command exposes a dependency, an edit changes the meaning of an earlier observation, and a test rules out a proposed fix. An interaction transcript preserves these events together with their context. Replacing that transcript with compact state requires deciding which information later decisions will need. The consequences extend beyond the length of the next prompt.

SKILL.state makes this replacement an explicit runtime design [Badhe et al., 2026]. Persistent instructions, mutable structured state, and the current observation replace an expanding conversation. The attraction is clear: when those inputs are bounded, context need not grow with the trajectory. But a bound on context is not a bound on the work required to solve the task. A smaller representation may omit evidence, require additional state-writing output, or change how the agent acts and stops. Cache-aware pricing further separates prompt size from monetary cost.

We study this tension in a terminal-based coding agent. The evaluation transfers the state-based runtime idea to a different model, agent, and benchmark; it is not a reproduction of the original study. Our central question is what information must survive history replacement for compression to remain useful over a complete task.

The comparisons separate decisions that are easily conflated under agent memory. First, who writes the memory? A model can maintain a semantic account of its work, while the runtime can retain observed tool events directly. This motivates hybrid state: execution records and model-written interpretation have different owners. Second, what does the runtime retain?

File references, modifications, command outcomes, and diagnostics convey different amounts of execution evidence. Third, which original interactions remain available? A recent-history window preserves detail without requiring the state writer to anticipate its relevance. Finally, the update protocol and schema determine how the model expresses its semantic memory.

The results connect these design choices to task success and resource use. Model-written state substantially reduces peak prompts while lowering success and increasing conditional successful-execution cost. An exploratory component comparison shows why hybrid memory cannot be treated as a single intervention: richer execution records perform better than file references alone, but additional records do not yield continuous improvement. Retaining recent interactions produces the clearest recovery in both state families. The tested schema and protocol changes do not.

These findings motivate evaluating structured state together with its retained evidence and surrounding history. We distinguish partial recovery from a complete replacement for history, and separate measured results from proposed adaptive switching or archival retrieval. The practical outcome is a set of quality–cost trade-offs, not a universal recommendation to discard or retain every transcript.

2 Context policies and research questions

History and state differ in what they preserve. History retains records in their original order. State rewrites selected details into fields that describe the current task. A state field may record that a test failed without preserving its complete output or the edits preceding it. Conversely, keeping those edits does not guarantee that a model will use them effectively.

There are also different ways to produce state. A model can maintain semantic content such as hypotheses and pending work. A runtime can record facts directly observable as the agent works, such as completed commands and their outcomes. We call a state that combines these sources *hybrid state*. Runtime-derived records reduce the need to restate observable events, but do not by themselves explain what those events mean or what the agent should do next.

The published SKILL.state contract jointly emits a state patch and an action [Badhe et al., 2026]. A separate state-update tool changes that interface: the agent updates memory through a tool call within its main loop. Our inline policy emits the patch and action together, while the schema comparison tests how the patch is organized. These changes test specific aspects of the port, not every way in which it differs from the published system.

We first ask whether prompt compression remains beneficial after accounting for task success, cumulative generation, and cost. We then examine what the runtime should record, whether recent interactions recover capability beyond those records, and whether schema or protocol changes offer an alternative remedy. Each comparison uses a within-family control. Adding recent history to state tests the value of that history within a state-based agent; it does not isolate the value of the state block itself.

3 Context Policies and Update Protocols

Let P denote persistent task instructions, o_t the current observation, and $H_{<t}$ the retained interaction records preceding it. The record includes observable messages and tool results; the notation makes no assumption about access to hidden model reasoning. A full-history policy constructs

$$C_t^{\text{full}} = [P, H_{<t}, o_t]. \tag{1}$$

For structured state Σ_t, the replacement policy is

$$C_t^{\text{state},k} = [P, \Sigma_t, \tau_k(H_{<t}), o_t], \qquad \tau_0(H) = \varnothing, \tag{2}$$

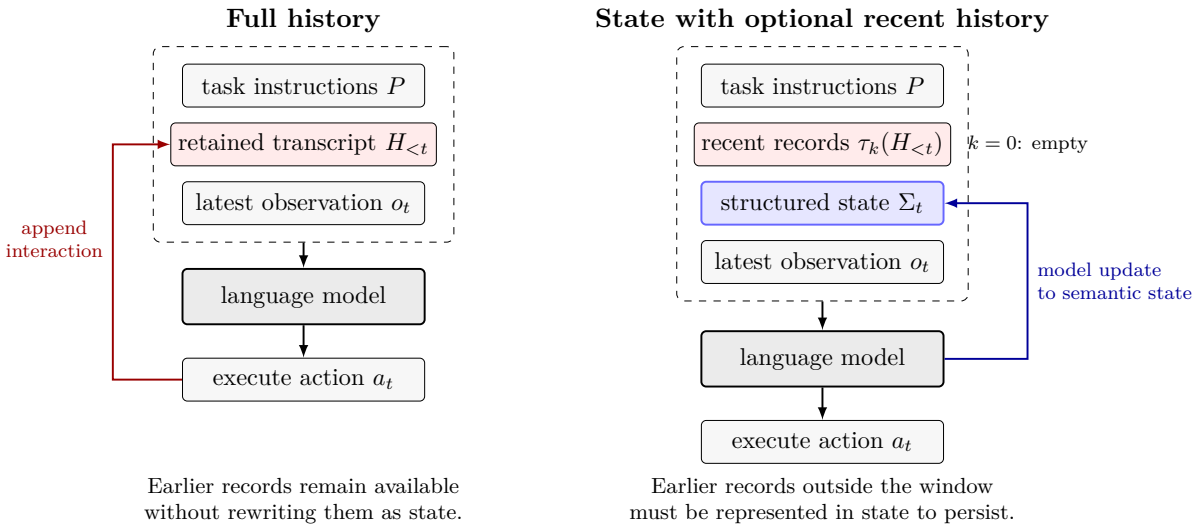


Figure 1: Context policies. State replaces the older transcript, while the retention parameter k controls the recent suffix. The arrows show information flow, not the number of model calls.

where τ_k selects the configured recent-history window, clipped at the start of the task. Full history is a separate reference policy, not state with an unlimited window. Figure 1 shows the information retained by these policies.

3.1 Who writes the state?

In model-written memory, the model maintains semantic state S_t through explicit updates. Hybrid memory separates this content from a runtime record:

$$\Sigma_t = (S_t, V_t), \qquad S_{t+1} = \text{Apply}(S_t, \Delta S_t), \qquad V_{t+1} = \text{Record}(V_t, a_t, r_t). \tag{3}$$

The recorder derives entries from observed tool events r_t; the model cannot edit those entries through its semantic patch. File references identify observed artifacts. Modification records describe changes reported by the tools, and command records preserve observed invocations and outcomes. Richer variants add diagnostic, environment, test-related, and failed-attempt records. These records let the system retain some evidence without asking the model to generate a new description of it.

The semantic state stores goals, hypotheses, interpretations, and pending work. For example, a command result belongs in V, while the hypothesis that it explains a failure belongs in S. This division assigns responsibility for maintaining information; it does not make V a complete or infallible account of the environment. Unobserved changes can be missed, and recognizing a test outcome from command text can require heuristics. Recent history is an independent addition to either state family. We reserve "hybrid" for the ownership distinction.

3.2 How is an update coupled to an action?

A separate update tool lets the model submit a state patch through the tool interface. An inline protocol instead asks for a joint envelope containing $\{\Delta S_t, a_t\}$, following the form of the published SKILL.state contract [Badhe et al., 2026]. These protocols differ in the relationship between state maintenance and environment actions; a tool-mediated policy step need not correspond to a single model invocation.

Algorithm 1 specifies the context and ownership abstraction used to distinguish the policies. Its PolicyStep leaves message serialization and the number of tool-mediated calls unspecified; with full history it emits an action and leaves S unchanged. Algorithm 2 describes the joint

patch–action contract, including rejection before action execution. It is a reference contract for the comparison, not a recovered execution trace or a claim that every evaluated implementation obeyed every detail. A fixed retry count or automatic fallback to full history is not assumed.

Algorithm 1 Context selection and state ownership

Require: Instructions P, policy π, retention setting k, update protocol q, initial observation o

Initialize history H, semantic state S, runtime record V
while the task has not terminated **do**
    if $\pi =$ FullHistory **then**
        $C \leftarrow [P, H, o]$
    else
        $\Sigma \leftarrow S$ **or** (S, V) according to state ownership
        $C \leftarrow [P, \Sigma, \tau_k(H), o]$
    end if
    $(S', a) \leftarrow \text{PolicyStep}(C, S, \pi, q)$
    $r \leftarrow \text{Execute}(a)$; $S \leftarrow S'$
    if hybrid memory is enabled **then**
        $V \leftarrow \text{Record}(V, a, r)$
    end if
    Append a record of (o, a) to H; $o \leftarrow r$
end while

Algorithm 2 Inline patch–action contract

Require: Current context C and semantic state S

$(\Delta S, a) \leftarrow \text{Model}(C)$
if $\text{Valid}(\Delta S, S)$ **then**
    return $(\text{Apply}(S, \Delta S), a)$
else
    Keep S unchanged and return a validation observation
    Do not execute a from the rejected envelope
end if

3.3 Context size and complete-task cost

If interaction records have bounded size and the number of model calls per action is bounded, retaining the full transcript gives prompt growth of order t and cumulative input growth of order T^2 over T actions. A state policy has an action-independent context bound only when the state, observation, and retained-history records are all bounded. Under those assumptions and fixed k, cumulative input grows as $O(T)$. Neither statement fixes how many actions the policy will take to finish.

Let U, K, and O be accumulated uncached-input, cache-read, and output tokens. With their respective tariffs α, γ, and β, normalized cost is

$$D = \alpha U + \gamma K + \beta O. \tag{4}$$

A smaller maximum prompt does not determine any of these task-level sums. State updates can add output, and a changing prompt can alter cache reuse; these are possible explanations to examine, not consequences proved by the context-size bound. The evaluation therefore reports success, prompt size, token categories, and cost separately.

4 Experimental methodology

4.1 Agent, tasks, and comparisons

The evaluation uses `zai/glm-5.3-flash`, Pi coding agent version 0.85.1, and the Harbor 0.22 release family. Tasks come from Terminal-Bench 2.0, which supplies executable environments and task-specific verification tests [Merrill et al., 2026]. The agent works through coding and terminal tools. Success is the binary task-verifier outcome, and the recorded execution limit is 200 turns.

The analyzed subset contains 87 tasks; two further benchmark tasks were excluded because their environments could not be initialized before agent execution. The model-written comparison has four conditions with three repetitions per task. The final hybrid comparison has eight conditions with two repetitions per task. An exploratory component comparison covers four runtime-record configurations and a full-history control with one repetition on the same tasks. Its control and richest configurations overlap the first repetition of the final hybrid comparison; they are not an independent replication. Each comparison uses its own full-history reference. An observation is a task execution under a condition and repetition, and contrasts match the same task and repetition. We do not pool the controls or treat repeated executions as additional tasks.

The model-written conditions use a separate state-update tool and retain recent history at settings $k = 0, 2, 6$. The hybrid reference combines runtime-derived records, including failed attempts, with model-written semantic content. Its variants remove failure records, retain recent history, or cross tool versus inline updates with an existing versus task-adapted schema. The reference condition is shared by these hybrid comparisons. The adapted schema was developed using experience from the same task set, so that comparison is exploratory and in-sample.

The exploratory component comparison progressively groups file references, modifications and commands, diagnostics and environment and test records, and failed-attempt records. These are comparisons of bundled representations, not isolated effects of every field. The weaker configurations were discontinued after preliminary performance was observed. Their results describe the design search and do not constitute a confirmatory component study.

The parameter k denotes the configured retention setting. The available records do not resolve individual messages versus complete exchanges consistently across implementations. We therefore do not interpret equal values of k as a verified equality of serialized context. The formal model describes information sources and update order; it does not specify hidden reasoning retention or an exact API message layout.

4.2 Outcomes and resources

For a condition a, task i, and repetition r, let Y_{iar} denote binary success. With n tasks and R repetitions, the solved rate is

$$\widehat{s}_a = \frac{1}{nR} \sum_{i=1}^{n} \sum_{r=1}^{R} Y_{iar}. \tag{5}$$

Differences in solved rate are percentage-point differences against the named reference within the same comparison.

Token measures distinguish peak prompt size, the maximum input context of a model call, from cumulative uncached input, cache reads, and output over the execution. The model-written comparison uses the fixed reference tariff

$$D = 10^{-6}(0.15U + 0.03K + 0.50O), \tag{6}$$

where U, K, and O denote uncached input, cache-read, and output tokens. Costs are in US dollars under this convention, not provider invoices. Recorded costs in the exploratory component

comparison agree with this tariff to rounding precision. The final hybrid comparison supplies reported token and cost medians; its pricing calculation and per-call prompt peaks cannot be reconstructed from the available aggregates. We therefore compare monetary values within each comparison and do not interpret them as verified provider invoices or cross-comparison price differences.

We report median execution cost and cost per observed success, $\sum D / \sum Y$, over retained executions, including failures. Its reciprocal is successes per dollar. Successes per million tokens use total recorded input, cache-read, and output tokens as the denominator. These metrics describe the retained workload, excluding discarded attempts and unretained retries. They do not estimate the cost of retrying until success.

A second resource comparison is conditional on both policies solving the same task in the same repetition. For each resource, we report the median of within-match condition/reference ratios. The selected subset differs by condition. For the exploratory component comparison, a sensitivity analysis also fixes the subset to tasks solved by every included configuration. This estimand describes relative resource use on shared successes, whereas cost per success also reflects failures and differences in capability. A ratio of marginal medians is not substituted for a median of matched ratios.

4.3 Statistical analysis

Solved-rate tables report two-sided exact McNemar p-values for the named matched contrasts. Conditional cost-ratio tables use a two-sided exact sign test against a ratio of one, excluding ties. These are unadjusted, exploratory reference tests under an independence assumption for matched observations. Repeated tasks and possible execution dependencies limit that interpretation. The tests do not adjust for within-task dependence or control the familywise error rate.

For the model-written comparison, pointwise task-bundle bootstrap intervals provide a complementary assessment of solved-rate differences. We resample task identities with all their conditions and repetitions, using 100,000 resamples and seed 20260922. Percentile endpoints define the 95% intervals. This preserves the observed within-task structure under a task-resampling interpretation, without establishing that the benchmark is a random sample. The hybrid aggregates do not retain the task-level covariance needed for the same analysis. Small differences or non-significant tests are not interpreted as equivalence.

The supplementary horizon analysis partitions tasks by the full-history control's median recorded turns, T, at $T = 11$. Membership is fixed across state conditions. This post-hoc stratification describes shorter and longer reference trajectories; it does not measure a causal effect of horizon.

4.4 Analysis material

The model-written results are recomputed from released execution-level records using their canonical inclusion rules. The exploratory component results likewise use execution-level records, while the final hybrid results use condition summaries and matched discordant counts. This distinction sets the resolution of the analyses: the latter support the reported contrasts but not reconstruction of individual trajectories. The reproducibility appendix specifies the available material and derived metrics.

5 Compression and the cost of completing a task

Model-written state delivers the intended compression but does not preserve task success. Without recent history, its solved rate is 34.9%, compared with 61.3% for full history. On shared successes, the median peak-prompt ratio is 0.47, yet cost is 2.57 times the reference and output

use is 4.55 times higher (Table 1). The smaller input context therefore coexists with more expensive successful execution.

Table 1: Resource ratios on shared successes: median of matched condition/full-history ratios. Peak prompt is a per-call maximum; the token columns are cumulative. The exact sign test concerns cost ratios.

Approach	Cost	Peak prompt	Input	Output	Cached	p
Model-written state	2.57×	0.47×	1.86×	4.55×	0.89×	1.4×10^{-12}
State + history ($k = 2$)	1.39×	0.60×	1.62×	2.08×	0.40×	4.7×10^{-4}
State + history ($k = 6$)	1.15×	0.81×	1.96×	1.36×	0.21×	0.036

The token breakdown shows why prompt reduction cannot be used as a cost estimate (Table 2). Across retained executions, state alone has a smaller peak prompt but more recorded turns and higher output. Retaining recent history raises input volume while bringing output closer to the full-history level. Cache reads also change substantially across policies. Under the reference tariff, uncached input, cached input, and output have different prices, so neither peak context nor total token volume captures their monetary consequences.

Table 2: Median recorded turns T, per-execution peak prompt, and cumulative token use for model-written state policies. Token counts are in thousands; input excludes cache reads. Peak is the median of execution maxima, not the median per-call prompt. Token columns are separate marginal medians and cannot be summed to obtain median total tokens.

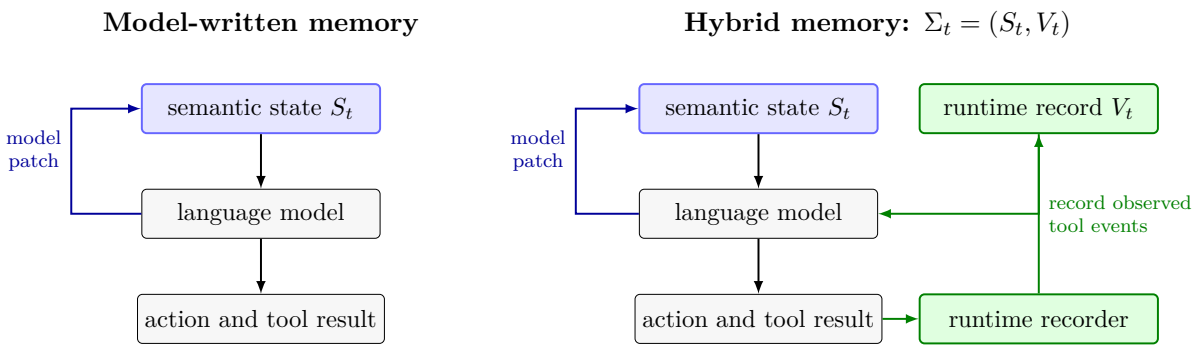
Approach	Median T	Peak prompt (ktok)	Input (ktok)	Output (ktok)	Cached (ktok)
Full history	13	16.1	28.8	16.4	87.4
Model-written state	25	4.0	24.9	37.4	36.7
State + history ($k = 2$)	14	6.2	30.3	20.1	19.1
State + history ($k = 6$)	11	11.3	53.3	16.6	14.8

Including failures does not reverse the state-only result: cost per observed success is \$0.1630, versus \$0.1116 for full history. This measures the retained workload, whereas the conditional ratios compare successful execution of the same task. Both expose costs that a prompt-size-only evaluation would miss.

The breakdown is consistent with several mechanisms. Generating state updates consumes output; incomplete memory may require additional inspection; changing the prompt can alter cache reuse. The aggregate measurements do not identify how much each mechanism contributes. They do establish the design problem: reducing active context is insufficient if the resulting policy loses task-relevant evidence or requires more work to finish. The next comparisons examine what the runtime can retain directly and what remains useful in the original interaction record.

6 What should the runtime remember?

Hybrid state separates observed execution records from model-written interpretation. The runtime can preserve an observed command outcome without requiring the model to rewrite it, while the model maintains hypotheses and pending work. Figure 2 shows this division. Its value depends on the contents of the runtime record: remembering which file was seen is different from retaining what was changed and which commands followed.



The model updates S; the runtime updates V.
A runtime record preserves captured evidence, not a complete account of the environment.

Figure 2: State ownership. Hybrid memory retains model-written semantic content and adds runtime-derived records. Recording an event does not establish that an extracted interpretation is correct or that all relevant events were captured.

6.1 From file references to execution evidence

The exploratory component comparison makes this distinction visible (Table 3). File references alone reach 28.7% solved. Including modifications and command records raises the observed rate to 40.2%, while median output falls from 23.2 to 16.5 thousand tokens. Median recorded cost falls from \$0.01406 to \$0.01016. The comparison changes a group of records, so it does not isolate which component accounts for the difference.

Table 3: Exploratory comparison of runtime-record contents, with model-written semantic state under the same schema. Success and efficiency use all retained outcomes; output and cost are per-execution medians. Rows beginning with + add to the preceding configuration. These are grouped design comparisons rather than independent estimates of individual components.

Runtime-owned information	Solved rate (%)	Median output (ktok)	Median cost ($)	Solved/$
Full history	55.2	16.4	0.01139	22.39
File references	28.7	23.2	0.01406	12.87
Files, changes, commands	40.2	16.5	0.01016	20.79
+ diagnostics, environment, tests	42.5	19.1	0.01316	20.92
+ failed-attempt records	41.4	18.6	0.01196	21.19

Adding diagnostics, environment, and test-related records gives 42.5%; adding failed-attempt records gives 41.4%. Neither reaches the full-history reference of 55.2%. The largest observed improvement is thus between file references and the representation that also records modifications and commands. Further enrichment produces a plateau with no monotonic improvement in success or cost (Figure 3).

Efficiency relative to the sparse representation improves as well. Successes per dollar rise from 12.87 with file references to 20.79 with modifications and commands. The richer configurations approach, but do not exceed, the full-history value of 22.39 on this workload. This is evidence for retaining execution information beyond artifact references, not evidence that hybrid state has replaced history at lower cost.

The conditional cost comparison is sensitive to which successes are included. On the same tasks solved by every configuration, the cost ratios for the two richest variants are 1.03 and 1.17 relative to full history. They remain far below the file-reference ratio of 2.50, but do not establish a cost advantage over the control. Appendix B reports this sensitivity analysis. The component results are exploratory and partly overlap the final comparison, as specified in Section 4.

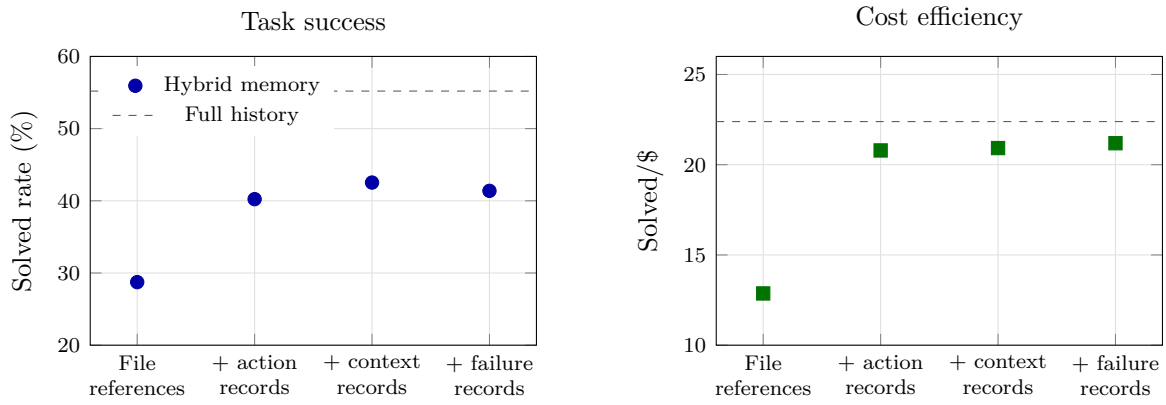


Figure 3: Exploratory comparison of cumulative runtime-owned information. Each position adds records to the preceding configuration: action records cover file changes and commands; context records cover diagnostics, environment, and tests. Efficiency includes the cost of unsuccessful episodes. Dashed lines show the same full-history reference. These descriptive observations share their richer configurations and reference with the first repeat of the final comparison; they do not isolate individual components or establish a monotonic benefit.

6.2 Do failure records close the remaining gap?

The final within-hybrid comparison tests the addition of failed-attempt records more narrowly (Table 4). Success rises from 38.5% without them to 39.7% with them, while the full-history control reaches 51.7%. Median cost falls slightly. The matched comparison provides little evidence of a large additional benefit from this representation of failures.

Table 4: Final hybrid comparison with and without failure records. Token use and cost are per-execution medians; token counts are in thousands. Exact McNemar tests compare each policy with full history.

Approach	Solved rate (%)	Output (ktok)	Cached (ktok)	Median cost ($)	p
Full history	51.7	16.4	62.5	0.0104	–
Hybrid, no failure records	38.5	19.1	10.2	0.0127	4.3×10^{-4}
Hybrid state	39.7	17.3	9.3	0.0120	0.001

The component comparison and the final failure-record contrast point to a limit of automatic recording. Richer execution evidence performs better than a sparse artifact index, but the tested record still does not substitute for the transcript. A stored outcome need not preserve the sequence or interpretive context needed by a later decision. This suggests retaining some original interactions alongside state. Adding more fields alone has not recovered the missing capability.

7 Recovering capability with recent history

A recent-history window preserves original interactions without requiring the state writer to select every relevant detail in advance. This is an independent change to either memory family. It adds a source of evidence without changing who writes the state.

For model-written memory, success rises from 34.9% without history to 45.2% at $k = 2$ and 59.0% at $k = 6$ (Table 5). The largest setting recovers much of the deficit relative to the 61.3% full-history control. The task-bundle interval for its difference is $[-7.7, +3.4]$ percentage points, compared with $[-34.1, -18.8]$ for state alone. Proximity of the point estimates does not establish equivalence.

Table 5: Recent-history retention, task success, and cost per observed success. Each family has its own full-history reference. Differences and exact McNemar tests use that reference; panels are not pooled. Cost per success includes retained failures.

Approach	k	Solved rate (%)	Cost/success ($)	Δ (pp)	p
Model-written memory					
Full history	–	61.3	0.1116	–	–
Model-written state	0	34.9	0.1630	-26.4	2.1×10^{-13}
Model-written state	2	45.2	0.1091	-16.1	9.7×10^{-7}
Model-written state	6	59.0	0.0768	-2.3	0.488
Hybrid memory					
Full history	–	51.7	0.0473	–	–
Hybrid state	0	39.7	0.0468	-12.1	0.001
Hybrid state	2	40.8	0.0462	-10.9	0.001
Hybrid state	6	48.3	0.0501	-3.4	0.392

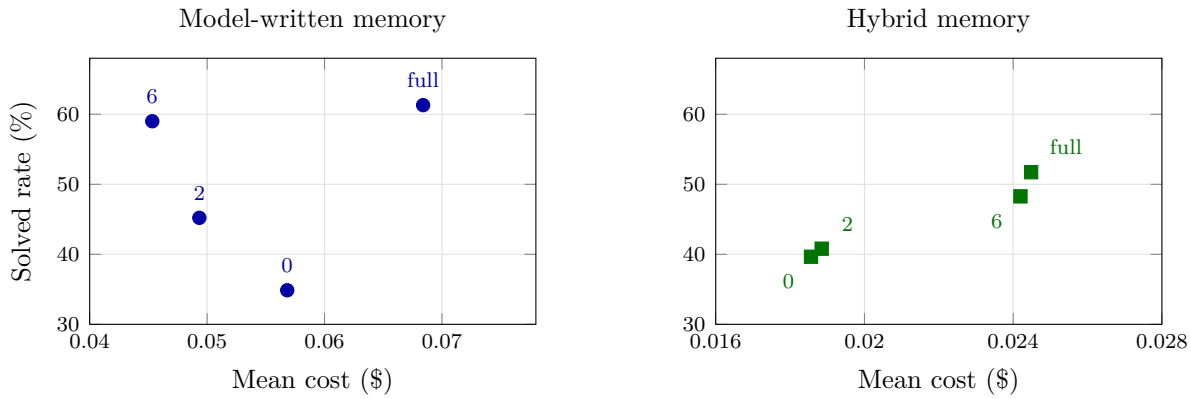


Figure 4: Success–cost trade-offs when retaining history. Labels are the retention setting k; full denotes the full-history reference without a state block. Each panel includes failures in mean cost and uses its own control. Hybrid mean costs use rounded reported totals. The horizontal scales differ.

Hybrid state shows the same direction of recovery. Its solved rate rises from 39.7% without retained history to 40.8% at $k = 2$ and 48.3% at $k = 6$. The largest setting improves on hybrid state by +8.6 percentage points ($p = 0.011$), against a full-history reference of 51.7%. Recent interactions improve the outcome even when the runtime already retains rich execution records.

7.1 Recovery has different economic consequences

For model-written memory with $k = 6$, cost per observed success is $0.0768, below the control's $0.1116. For hybrid memory with $k = 6$, it is $0.0501, above its own control's $0.0473. Figure 4 shows the resulting trade-offs. The histories improve success in both families, but do not confer the same economic advantage.

The model-written result is compatible with the conditional cost ratio of 1.15 in Table 1. Cost per success includes failures, termination behavior, and tasks solved by only one policy; conditional ratios include only shared successes. A policy can therefore have lower cost per observed success on the retained workload without being cheaper on the tasks both policies solve. The complete efficiency breakdown appears in Appendix C.

These results make recent-history retention the clearest measured partial remedy among the tested changes. They do not establish that structured state adds value over an equally sized history window: that requires a history-only control. Nor do they identify whether the

useful material is an omitted observation, action order, formatting, or another property of the interaction record. The supplementary trajectory-length comparison in Appendix D describes where the observed deficits occur.

8 Can schema or protocol changes replace history retention?

A state-based agent might lose capability because its schema fits the task poorly or because a separate update tool changes the intended runtime contract. The hybrid comparison tests these alternatives by crossing the update interface with the semantic schema while leaving recent history absent (Table 6).

Table 6: Update protocol and schema within hybrid state. Output and cost are per-execution medians; cost per success includes retained failures. Exact McNemar tests compare each configuration with Tool + Existing schema.

Protocol	Schema	Solved rate (%)	Output (ktok)	Median cost ($)	Cost/success ($)	p
Tool	Existing	39.7	17.3	0.0120	0.0468	–
Inline	Existing	35.1	28.6	0.0165	0.0656	0.185
Tool	Task-adapted	41.4	21.7	0.0137	0.0474	0.678
Inline	Task-adapted	28.7	33.1	0.0190	0.1008	0.001

The task-adapted schema changes success from 39.7% to 41.4%, a +1.7 percentage-point increase, while median cost rises from $0.0120 to $0.0137. This is a small observed gain, not a recovery comparable to retaining the larger history window. Its cost per success, $0.0474, is also slightly above the reference value of $0.0468.

Inline updates have a less favorable outcome. With the existing schema, success falls to 35.1% and median cost rises to $0.0165. Combining inline updates with the adapted schema gives 28.7% success and $0.1008 per observed success. The descriptive protocol–schema difference-in-differences is −8.0 percentage points: the inline effect is more negative under the adapted schema. The available summaries do not support a task-level interaction test.

These comparisons do not support either tested change as a substitute for recent-history retention. They also show why matching the shape of a published patch–action interface is insufficient evidence of matching its behavior in a different agent. Inline output changes the relationship between state maintenance and actions, and its output-token median is higher in both schema conditions. The measurements do not isolate patch content, invalid updates, or another trajectory change as the cause. Better schemas or implementations remain possible; the specific interventions tested here do not remove the deficit.

9 Implications for memory design

9.1 Preserve evidence, not only a compact account

The results support the motivation for structured memory while identifying where the tested replacements fail. Smaller peak prompts demonstrate that the active record can be compressed. The accompanying success and cost deficits show that compression quality cannot be judged from its size. A memory policy must be evaluated through what the agent can still do with it.

Hybrid ownership offers a concrete way to separate two responsibilities. The runtime retains observed execution evidence; the model maintains its interpretation and next steps. The component comparison suggests that recording modifications and commands is more useful than remembering file references alone. It does not show that every added record improves the agent,

or that the richer representation is sufficient. This distinction matters when choosing a schema: a larger collection of fields is not necessarily a better record of the task.

9.2 Retain a path back to the interaction

Among the tested changes, recent-history retention gives the clearest recovery. It preserves observations together with neighboring actions instead of requiring the state writer to identify their future relevance. This is a plausible explanation for its benefit, not an identified causal mechanism. The practical result is nevertheless direct: the tested compact state performs better when some original interactions remain available.

The choice depends on the objective. Model-written state with the larger history window combines a near-control success estimate with lower cost per observed success, but its confidence interval still permits a meaningful capability deficit. The corresponding hybrid policy recovers success without beating its control on that cost measure. Neither policy dominates full history on every relevant criterion. A deployment with a strict context budget may value a different point from one prioritizing completion probability or cost.

9.3 What would establish a stronger solution?

The evidence motivates selective preservation of evidence in both state and history. An adaptive policy could retain full history while trajectories are short and introduce state as the record grows. A retrievable archive could retain older evidence without keeping it in every prompt. These are design hypotheses, not evaluated features of the present system. Their thresholds, retrieval reliability, and complete-task costs require separate experiments.

A decisive next comparison would hold the recent-history window constant and remove the state block. It would determine whether structured memory adds value beyond the interactions already retained. A separate test of schema degradation would be needed to establish whether runtime-owned records reduce sensitivity to semantic schema design. The current task-adapted-schema comparison does not answer that question.

The measured result is therefore a partial repair with a clear boundary: richer execution records improve on a sparse representation in exploratory data, and recent interactions recover much of the observed capability loss. Full replacement of history, reliable adaptive switching, and recovery of discarded evidence remain open.

10 Threats to validity

Implementation and selection. The findings concern specific implementations, not structured memory in general. Complete execution-source snapshots, generation settings, and prompt traces are unavailable for independent replay. The model-written release specifies inclusion rules, but their prospective timing is not independently established. The hybrid collector description leaves budget-related exclusions ambiguous, so outcome-dependent selection cannot be ruled out. Conditions were not demonstrably balanced over execution time and provider route. The exploratory component comparison informed which configurations were retained, and its richest conditions overlap the final comparison. It must not be interpreted as independently confirmed component effects. Bundled additions also prevent attribution to individual record types.

Measurement and inference. Verifier-defined success measures benchmark completion rather than memory quality in isolation. The available retention parameter does not establish identical message semantics across implementations. Reference-tariff cost and reported cost are distinct accounting conventions, and neither includes all discarded attempts. Repeated tasks,

outcome-conditioned resource subsets, and exploratory multiple comparisons limit statistical interpretation. The reported intervals do not establish representative task sampling, and the hybrid aggregates do not support the same task-level uncertainty analysis.

Generalization. The evaluation uses one model and coding-agent configuration on a limited task collection. Schema adaptation used that same collection. Results need not transfer to other models, state writers, retrieval mechanisms, or workloads near a context-window limit. The missing history-only control prevents a claim that state and recent history are complementary. No validated trace-level analysis identifies a causal explanation for the observed deficits.

11 Related work

SKILL.state [Badhe et al., 2026] replaces append-only history with mutable task state. Its tests include both synthetic environments and interactive benchmarks. The present work studies transfer to terminal-based coding, with different model, runtime, and benchmark choices. Its negative results concern the tested systems and do not contradict the original results.

An imposed event horizon differs from an agent's chosen trajectory. An externally specified event stream lets researchers measure behavior as the number of events grows. In an interactive coding task, the policy also affects how many actions are taken before the agent stops. Lower context growth at a fixed horizon does not determine the cost or success of such trajectories.

Prompt compression offers another approach to reducing model input. LLMLingua [Jiang et al., 2023] uses budgeted, token-level compression to shorten prompts. Replacing history with state instead requires the agent to maintain selected details as explicit fields. The two approaches differ in what they preserve and how they encode it; this study does not compare them directly.

ReAct [Yao et al., 2023] interleaves reasoning and actions with environmental feedback. A context policy governs which earlier records remain available within such a loop. Our comparisons vary that policy without claiming that retained records alone determine how well the agent reasons. Terminal-Bench [Merrill et al., 2026] provides the executable tasks and verifier outcomes used to measure the resulting task-level behavior.

12 Conclusion

SKILL.state's context-management idea remains useful as a way to reduce the active record presented to an agent. In the tested coding setting, however, model-written state exchanges a smaller prompt for lower task success and higher conditional successful-execution cost. Context compression alone does not establish an efficient replacement for history.

The design comparisons identify partial remedies and their limits. Runtime-owned execution records make hybrid memory more informative than file references alone, but further enrichment does not consistently improve outcomes. Retaining recent interactions gives the clearest capability recovery in both state families. The tested schema and protocol changes do not offer comparable recovery, and the economic value of history retention differs between implementations.

Structured memory should therefore be judged by the evidence it preserves and the cost of completing the task with that evidence. Recent-history retention is a measured partial remedy, while the incremental benefit of state itself requires a matched history-only control. Adaptive switching and archival retrieval are motivated next steps, not solutions established by these results.

References

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. URL https://arxiv.org/abs/2608.26263. v3, 02 Sep 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023. doi: 10.18653/v1/2023.emnlp-main.825. URL https://aclanthology.org/2023.emnlp-main.825/.

Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868. arXiv:2601.11868. Version 2.0 used in this work.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

A Reproducibility and metric definitions

The analysis artifact contains execution-level records for the model-written policies and the exploratory runtime-record comparison, alongside final aggregate summaries for the hybrid policies. Table entries and numerical claims are generated from those inputs. The aggregate summaries contain solved-rate margins and matched discordant outcomes, sufficient to reconstruct the reported success contrasts and exact McNemar tests. They do not contain complete trajectories, per-call prompt measurements, or joint task-level outcomes for all configurations.

Resource ratios are computed separately for each condition against its own full-history reference, restricting to shared successes. For a resource X, the reported statistic is

$$\text{median}_{(i,r):\, Y_{iar}=Y_{i0r}=1} \frac{X_{iar}}{X_{i0r}}.$$

A zero reference denominator would make an individual ratio undefined. The analysis does not replace such a denominator with an arbitrary constant. Input and cached tokens are separate recorded counters.

The efficiency measures use all retained outcomes:

$$\text{Solved/\$} = \frac{\sum Y}{\sum D}, \qquad \text{Solved/Mtok} = \frac{10^6 \sum Y}{\sum (U + K + O)}.$$

The latter measures logical token volume under the recorded counter convention, not equivalence of computational work across cached and uncached tokens. Hybrid cost efficiencies are approximate because the available aggregate costs are rounded. Unavailable token totals and prompt peaks are not reconstructed from marginal medians.

B Sensitivity of exploratory component costs

The exploratory comparison permits a check of outcome-conditioned selection. A separate shared-success subset for each configuration changes the tasks contributing to its cost ratio. To compare the configurations on the same tasks, we additionally restrict all ratios to the intersection of successes across every included configuration and the control. Table 7 reports both calculations.

Table 7: Exploratory resource sensitivity. Variable-subset cost ratios use each configuration's shared successes with full history. Fixed-subset ratios use the same all-configuration success intersection. Entries are medians of task-wise ratios to full history, not ratios of marginal medians.

Runtime-owned information	Variable subset Cost	Fixed subset Cost	Fixed subset Turns	Fixed subset Output
File references	2.31×	2.50×	1.61×	4.03×
Files, changes, commands	1.20×	1.07×	0.77×	1.56×
+ diagnostics, environment, tests	0.92×	1.03×	0.75×	1.75×
+ failed-attempt records	0.94×	1.17×	0.79×	1.73×

The two richest representations have variable-subset cost ratios of 0.92 and 0.94, but fixed-subset ratios of 1.03 and 1.17. The large reduction relative to file references remains. A cost advantage over full history does not. The fixed subset is still selected on outcomes, so this sensitivity check describes successful-task resource use rather than an unconditional causal effect.

C Complete efficiency breakdown

Table 8: Cost and efficiency over retained executions, including failures. Successes per dollar use aggregate cost and observed successes, not ratios of medians. Token efficiency is reported where cumulative totals are available. Final hybrid efficiencies use rounded reported cost totals.

Approach	Median cost ($)	Mean cost ($)	Solved/$	Solved/Mtok
Model-written memory				
Full history	0.0137	0.0684	8.96	0.421
Model-written state	0.0239	0.0568	6.14	1.245
State + history ($k = 2$)	0.0151	0.0493	9.16	1.964
State + history ($k = 6$)	0.0154	0.0453	13.02	2.718
Hybrid memory (efficiencies approximate)				
Full history	0.0104	0.0245	21.13	–
Hybrid state	0.0120	0.0186	21.36	–
Hybrid + adapted schema	0.0137	0.0196	21.11	–
Hybrid + inline	0.0165	0.0230	15.25	–
Hybrid + inline + adapted schema	0.0190	0.0290	9.92	–
Hybrid + history ($k = 2$)	0.0122	0.0189	21.65	–
Hybrid + history ($k = 6$)	0.0144	0.0242	19.95	–

Table 8 supplements the cost-per-success comparisons in the main text. Median execution cost and aggregate cost per success answer different questions: the latter also includes the cost of failures and the shape of the cost distribution. Neither estimates the cost of repeated attempts until success.

D Success by reference trajectory length

Table 9 separates tasks using the full-history control's median recorded trajectory length. All state policies use the same partition. The larger state-only deficit appears in the longer-reference subset, while recent-history retention improves the observed solved rate in both subsets. Because the partition is post-hoc and trajectory length also reflects task characteristics, this pattern does not identify a causal horizon effect or a context-window crossover.

Table 9: Solved rate by the full-history control's median recorded turns T. Differences and exact McNemar tests use full history within the same subset. The grouping is descriptive and held fixed across conditions.

	Control median $T \leq 11$			Control median $T > 11$		
Approach	Solved rate (%)	Δ (pp)	p	Solved rate (%)	Δ (pp)	p
Full history	56.3	–	–	66.7	–	–
Model-written state	39.3	-17.0	6.6×10^{-5}	30.2	-36.5	1.7×10^{-9}
State + history ($k = 2$)	45.2	-11.1	0.011	45.2	-21.4	4.2×10^{-5}
State + history ($k = 6$)	55.6	-0.7	1.000	62.7	-4.0	0.473